

PROJECT PROPOSAL · 课程项目开题

基于长程对话的结构化记忆与 动态状态解析大模型问答系统

解决历史状态失效与过期召回，实现精准可靠的长程多轮问答

TEAM

6767组

MEMBER

陈易 许子萌 鲁瑞特 张崇洋
于国庆

DATE

2026/10/09

汇报人：陈易

CONTENTS

汇报目录

01 问题背景与目标范围

02 技术方案与系统架构

03 评测体系与功能路线

04 团队分工与协作保障

长程对话历史状态持续演化，普通向量检索极易引发过期失效

长程对话的三大核心痛点

- **状态持续变化**：多会话间用户工作、居住、计划与偏好持续更新（如从准备申请到通过面试）
- **新旧难以区分**：向量检索按语义相似度召回，同时取出当前状态与已失效历史状态
- **决策隐蔽出错**：基于过期状态生成错误答案，错误往往隐蔽难以直接察觉

本项目基于 LoCoMo 数据，开发一个支持结构化记忆构建、查询驱动的动态状态解析、有效记忆检索与可视化的 Web 应用。

系统在回答问题前先判断历史记忆之间的保留、更新替代与冲突关系，确定当前仍然有效的记忆，再据此生成答案，并在网页端展示记忆、原始证据与状态演化过程。

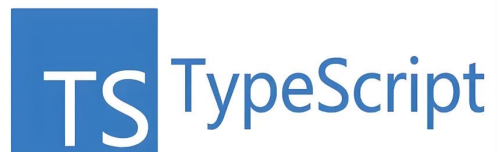
Vision

- 可解释的长程对话记忆管理原型，把“当前有效的记忆”显示搜索、判断并展示，同时量化记忆更新对问答质量的增益

In Scope

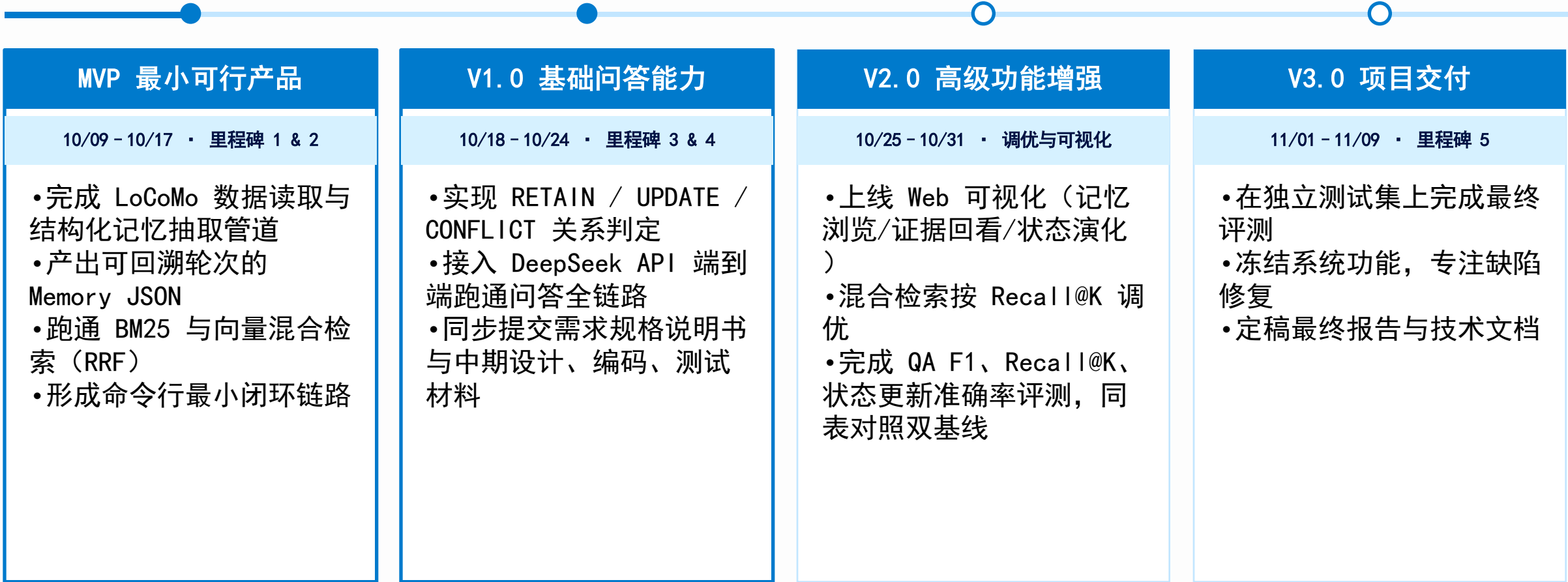
- 结构化记忆构建：抽取人物、事件、时间、状态导出为 JSON，严格保留原始对话轮次出处
- 动态状态解析：判定记忆间 RETAIN / UPDATE (SUPERSEDE) / CONFLICT 关系，锁定有效记忆
- 有效检索与问答：多路检索候选记忆 → 状态解析过滤 → 调用 DeepSeek API 生成最终答案
- Web 演化可视化：开发轻量 Web 界面展示结构化 Memory、原始证据与状态演化全过程
- 系统评测与基线对比：三指标严格评测，与 Full Text、Vector RAG 两种基准全面对照

02 · 技术栈



- 前端：Vue3 + TypeScript + Vite, 图表用 EChart
- 后端：Python
- 后端数据存储：SQLite存储问答与评测记录
- 工程与规范：Git + GitHub Flow, pytest进行测试, pre-commit进行git审查

03 · 功能路线图



陈易

项目管理与集成架构

- 计划与追踪
- 仓库与权限
- 架构决策
- 集成与评测接口
- 演示环境

鲁瑞特

数据、记忆与检索基础设施

- Memory 结构
- 结构化抽取
- JSON 导出
- 检索接口

于国庆

动态状态解析

- 关系判定
- 有效记忆选择
- prompt 与答案生成

许子萌

Web 与可视化

- 记忆与证据视图
- 状态演化展示
- 演示素材

张崇洋

评测与文档

- 三指标评测
- 对照实验
- 文档定稿
- slides 排版

代码管理与版本控制

- **代码托管**：统一基于 GitHub 仓库协作，严格遵守规范的 Git Conventional Commit 格式
- **分支策略**：主分支保持随时可发布，遵循 Github workflow
- **审查合并**：开发测试完成后发起 PR / Merge Request，至少一名成员 Code Review 后合入

接口与文档

- **接口契约先行**：严格遵循 docs/interfaces.md 定义 Python 接口，解耦 Web 层与算法层
- **知识库管理**：采用语雀知识库沉淀软件需求规格说明书、设计文档与实验日志
- **任务跟踪**：使用 GitHub Issues 统一收集 Bug 反馈、功能需求并分配负责人

日常沟通与进度同步

- **即时沟通**：微信群进行日常高频沟通与阻塞问题即时反馈
- **定期对齐**：每周定期线下/线上例会，同步进展并进行接口联调

风险与应对策略

风险 1 · 状态关系判定效果不达预期：关键路径成员只做核心算法，不承担辅助工作；先跑通机制再调优效果

风险 2 · 检索召回不足放大下游错误：设计多路检索并做去重合并，用 Recall@K 单独监控检索环节

风险 3 · 6 周内完成五个里程碑时间紧：每个里程碑两轮迭代 + 独立缓冲；功能优先跑通端到端

风险 4 · 课程评测所用对话不公开：严格不使用 gold evidence 与答案，规则保持通用

Thanks

6767组